Zack's IT Help Desk: The Azure Migration

From Self-Managed RAG Prototype to Cloud-Native, Zero-Secret Architecture

Applied AI Project Case Study

Zachary Ziernicki

Version 1.0 · July 2026

github.com/zziernicki-ui/ithelpdesk

Live: zacks-help-desk-byakb6ayhag2g9e0.centralus-01.azurewebsites.net

Python · FastAPI · Azure AI Search · Azure OpenAI · GitHub Actions · Application Insights · Managed Identity · React

At a Glance

	v1 — Prototype	v2 — Azure-Native
Deployment	Manual, undocumented steps	git push — GitHub Actions builds and deploys; failed builds never ship
Retrieval	In-process Chroma index, rebuilt on every restart	Azure AI Search: persistent index, hybrid keyword + vector search, semantic re-ranking
Knowledge base	Files inside the repository; update = redeploy	Blob Storage container; update = upload a file
Generation	Anthropic API, outside the cloud boundary	gpt-5-mini on Azure OpenAI, inside the Azure boundary
Authentication	Long-lived API keys in configuration files	System-assigned Managed Identity + RBAC; zero standing secrets
Observability	Raw log stream	Application Insights: OpenTelemetry request, dependency, and exception traces
Ingestion code	Custom loader, chunker, and vector store modules	Deleted — absorbed by the AI Search indexing pipeline

Key notes: three custom modules deleted · zero API keys anywhere in the system · 50-document, four-format corpus · steady-state cost in single dollars at demonstration traffic · one deployment step: git push.

Executive Summary

This case study documents the complete re-platforming of a deployed retrieval-augmented generation (RAG) system onto Azure-native managed services. The starting point was the working application from a preceding case study project; a FastAPI back end with an in-process Chroma vector store, a React chat interface, and generation through the Anthropic API, deployed manually to Azure App Service. That system worked, but it carried its prototype architecture with it, like the search index lived in memory and rebuilt on every restart, the knowledge base shipped inside the repository, deployments were manual, there was no telemetry, and API keys sat in configuration files.

The migration replaced each self-managed component with the corresponding Azure service: GitHub Actions for continuous deployment, Application Insights for observability, Blob Storage for the knowledge corpus, Azure AI Search with integrated vectorization for retrieval, and an Azure-hosted model for generation. A final hardening pass then eliminated every API key from the system, replacing them with a Managed Identity authorized through Azure RBAC, with the one remaining sensitive configuration value moved into Key Vault.

The build did not go according to plan, and that adaption therein is the most instructive part. The model migration was redirected twice by forces that only exist in real clouds. A marketplace quota of zero blocked the intended Claude deployment, and a model retirement blocked the first fallback. And in later phases identity hardening produced a persistent authorization failure that survived every obvious fix, ultimately diagnosed by decoding the application's identity token over SSH and tracing role assignments to the wrong resource. Three source modules were able to be deleted outright - reducing the size of the

code significantly - because managed services filled those same gaps. The result is a live system in which a git push is the only deployment step, retrieval survives restarts, telemetry answers questions logs could not, and there is no key anywhere for an attacker to exploit.

Problem and Objective

Problem

A working prototype and a production system can run the same code and still be different things. The v1 helpdesk assistant answered questions correctly, but its architecture made promises only a prototype can keep. The vector index was rebuilt from scratch on every process restart, producing a cold-start delay the interface literally apologized for. The knowledge base lived in the repository, so updating an article meant redeploying the application. Deployment itself was a manual step, unrepeatable and undocumented. When something failed in the cloud, the only diagnostic was a raw log stream. And the system authenticated to its model provider with a long-lived API key — the single most common cause of real-world cloud breaches.

Objective

Make the project exist entirely on Azure, using the platform's managed services the way an enterprise team would: automatic deployment on every push, production telemetry, a persistent cloud search index fed from cloud storage, generation from a model hosted inside the Azure boundary, and — as the final bar — zero standing secrets.

Technical Architecture

The final system centers on the same App Service, but everything around it changed. Code reaches production through GitHub Actions, which builds the React front end and deploys on every push to main. Retrieval is a query against Azure AI Search rather than an in-process database: the knowledge corpus lives in Blob Storage, and an AI Search indexer with integrated vectorization automatically chunks each document and embeds it with an Azure OpenAI embedding model. Generation calls gpt-5-mini through Azure OpenAI. Every hop authenticates through the App Service's system-assigned Managed Identity, authorized by RBAC role assignments; Application Insights receives OpenTelemetry traces from the FastAPI process; Key Vault holds the one remaining sensitive configuration value, injected through a Key Vault reference.

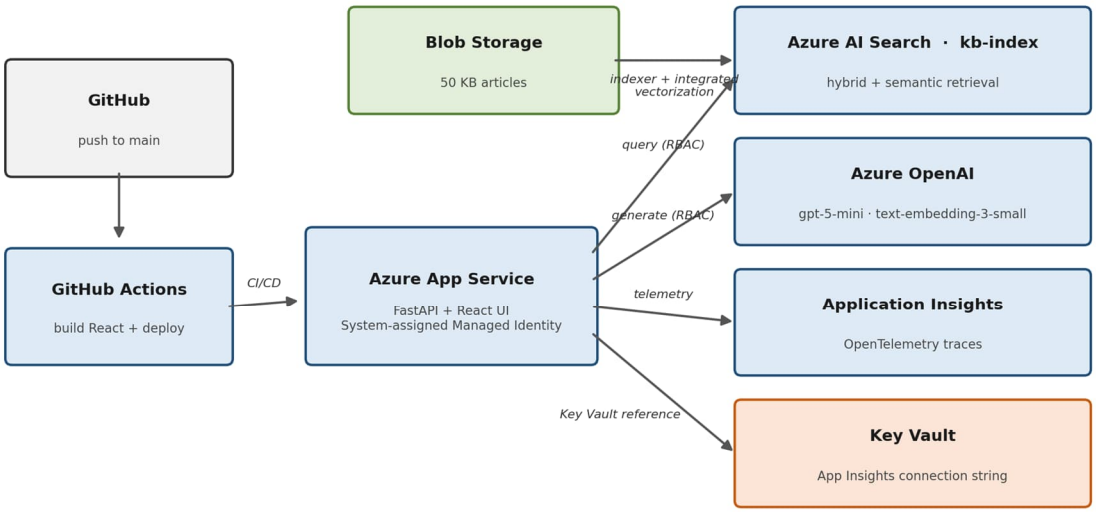

Figure 1. The migrated architecture: GitHub Actions deploys to App Service, whose Managed Identity authenticates every call to AI Search, Azure OpenAI, and Key Vault; Blob Storage feeds the AI Search indexer.

The deepest architectural change is what the application no longer contains. Three modules from v1 - the multi-format file loader, the chunker, and the vector store wrapper - were deleted. Their jobs absorbed by the AI Search indexing pipeline; using keyword matching and vector similarity in one request and then re-ranked by AI Search's semantic ranker. The application code is smaller than it was before the migration, while the system around it does strictly more.

Migration Workflow

The migration was planned as independent phases, each ending with a verifiable checkpoint so the live system was never more than one step from a known-good state. Development continued the AI-assisted co-working model of the previous projects — plans, diagnosis, and code drafted with Anthropic's Claude, every change reviewed, applied, and verified by hand against the live system.

Phase 1 — Continuous deployment

Azure's Deployment Center generated a GitHub Actions workflow connected to the repository, and one modification made it correct for this project: a Node build step so the React front end compiles in the pipeline rather than being committed pre-built. From this phase forward, every subsequent change in the migration reached production by git push, and the workflow's build log became a diagnostic surface in its own right.

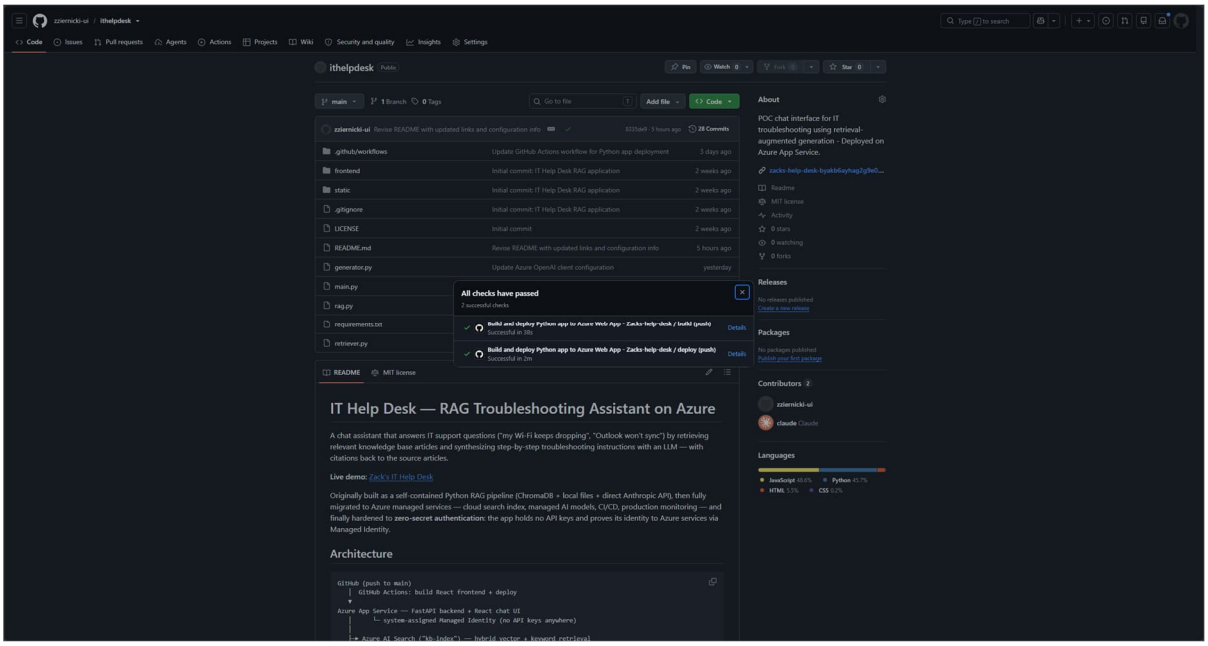

Figure 2. The delivery pipeline in effect: both workflow jobs — build (with the React frontend compile) and deploy — passing on the latest push to main.

Phase 2 — Observability

Application Insights was wired in through the OpenTelemetry distribution — two lines of Python and a connection string in App Service settings. The phase supplied a compact education in configuration failure modes: a deployment Correlation ID mistaken for the connection string, then a truncated paste that failed the parser, then an unguarded initialization that crashed local development where no connection string exists. The fix for the last — initialize telemetry only when the connection string is present — is the pattern production codebases actually use. The investment repaid itself for the rest of the migration: from this point on, failures were diagnosed from structured exception traces in the Failures blade rather than by scrolling raw log streams.

Phase 3 — The knowledge base becomes data, not code

The sample knowledge base was expanded to fifty single-article files spanning four formats — CSV, JSON, TXT, and PDF — deliberately mirroring how helpdesk knowledge actually accumulates, and uploaded to a Blob Storage container.

Phase 4 — Retrieval moves to Azure AI Search

An Azure OpenAI embedding deployment (text-embedding-3-small) and AI Search's 'Import and vectorize data' pipeline replaced the entire custom retrieval stack. The service generated the data source, the chunking skillset, the index, and the indexer — the work of v1's loader, chunker, and vector store, now all contained within one ecosystem. The rewritten retriever issues one hybrid query with semantic re-ranking, and three v1 modules were deleted in the same commit.

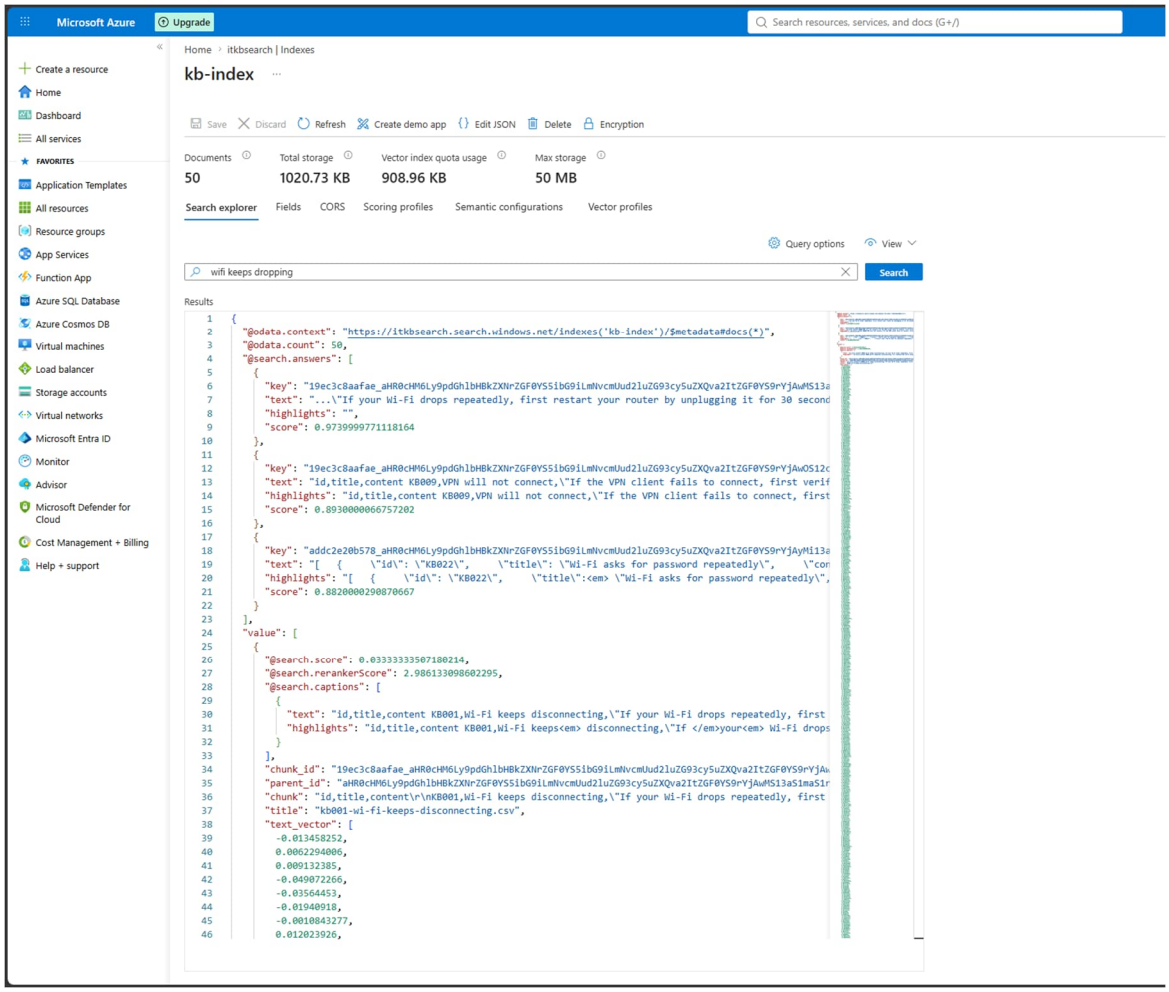

Figure 3. Search Explorer against the live kb-index: 50 documents, hybrid results for 'wifi keeps dropping' with semantic reranker scores, and the wizard-generated fields (chunk, title, text_vector) that the rewritten retriever maps.

Phase 5 — The model migration that required two pivots

The plan targeted Claude on Microsoft Foundry — the same model family as v1, newly GA on Azure infrastructure, promising a minimal code change. Reality intervened at the deploy dialog: the subscription's Claude quota was zero, a marketplace constraint on the account type, with a quota request as the only path through. Rather than stall the migration, the decision was made on the spot to pivot to Azure OpenAI on the existing resource. The first fallback, gpt-4o-mini, was then refused by the platform for a different reason entirely as the model had been retired for new deployments. The deployment that succeeded was gpt-5-mini — which, as a reasoning model, required a change from the max_tokens parameter in favor of max_completion_tokens and required a larger token budget because its internal reasoning spends from the same allowance. Each failure was a legitimately different lesson: quotas gate what you may deploy, lifecycles gate what still exists, and API contracts shift under model generations. The generator's structure (prompt, context assembly, citations) survived all three unchanged.

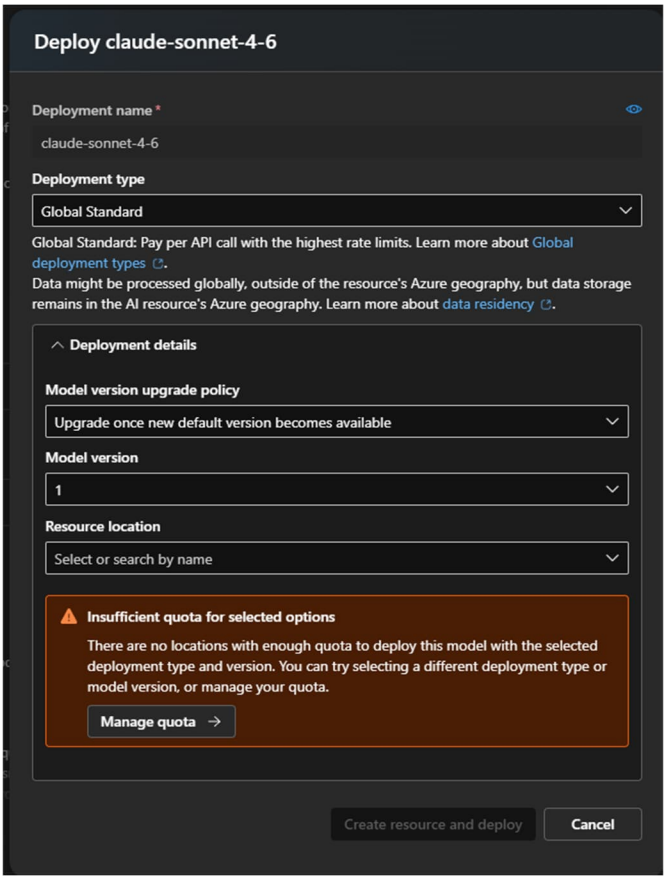

Figure 4. The pivot's cause, verbatim: the Claude deploy dialog reporting insufficient quota for every location and version.

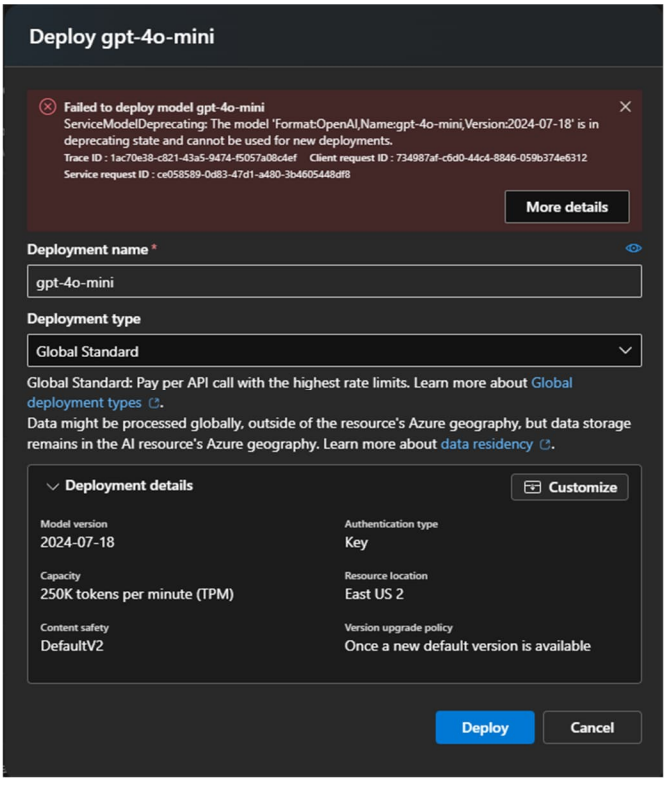

Figure 5. The second pivot: gpt-4o-mini refused as a deprecating model — retired four months before this deployment attempt.

Phase 6 — Zero secrets: Managed Identity, RBAC, and Key Vault

The final phase replaced every API key with identity. The App Service received a system-assigned Managed Identity where RBAC roles authorized it against Azure OpenAI and AI Search and the Application Insights connection string moved into Key Vault, injected by reference. The design took an afternoon. But this phase would encounter errors that took a half-day session to troubleshoot.

The symptom was a 401 PermissionDenied from the model endpoint that persisted for hours — through role assignments that looked correct in every portal view, propagation waits, and restarts. The diagnosis started like a controlled experiment. A differential test — running the identical code locally under a developer identity holding the same roles — succeeded, isolating the fault to the app's identity specifically. Inspecting the environment revealed the first real finding: the model deployment wizard had silently created a second AI resource in a different region, and the application had been calling it all along while every role assignment pointed at the original resource. Re-scoping roles to the resource group closed that gap — but the 401 persisted. The second finding required going below the portal entirely: from an SSH session on the App Service, requesting the identity's token from the instance metadata endpoint to see exactly which principal the application presented. That test exonerated the identity (the object ID matched, and a CLI query bound to it confirmed the roles were attached), which redirected attention to authorization semantics on the Foundry-type resource — where an additional role grant, of the kind that resource type checks for model inference, finally resolved the failure. The last acts of the phase were cleanup with teeth: the old keys were deleted from configuration and then regenerated at the services, so anything still silently depending on them would fail loudly. Nothing did.

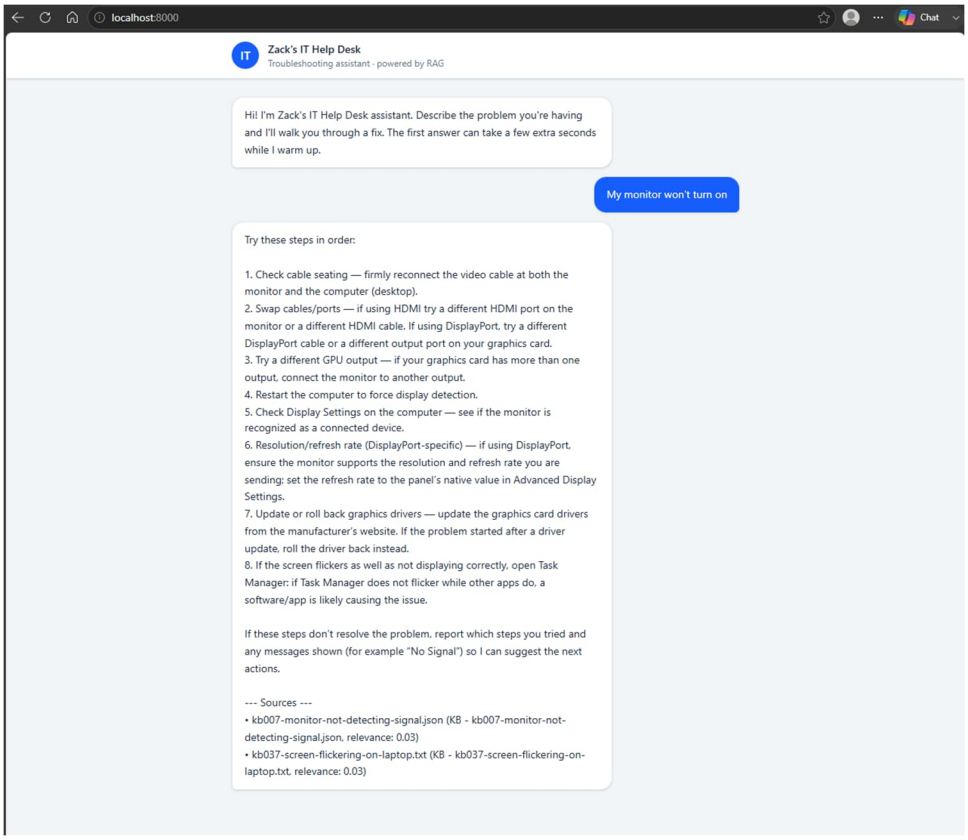

Figure 6. The differential test that isolated the fault: the identical keyless code answering on localhost under the developer's az login identity — proving the roles sufficed and narrowing the failure to the app's identity specifically.

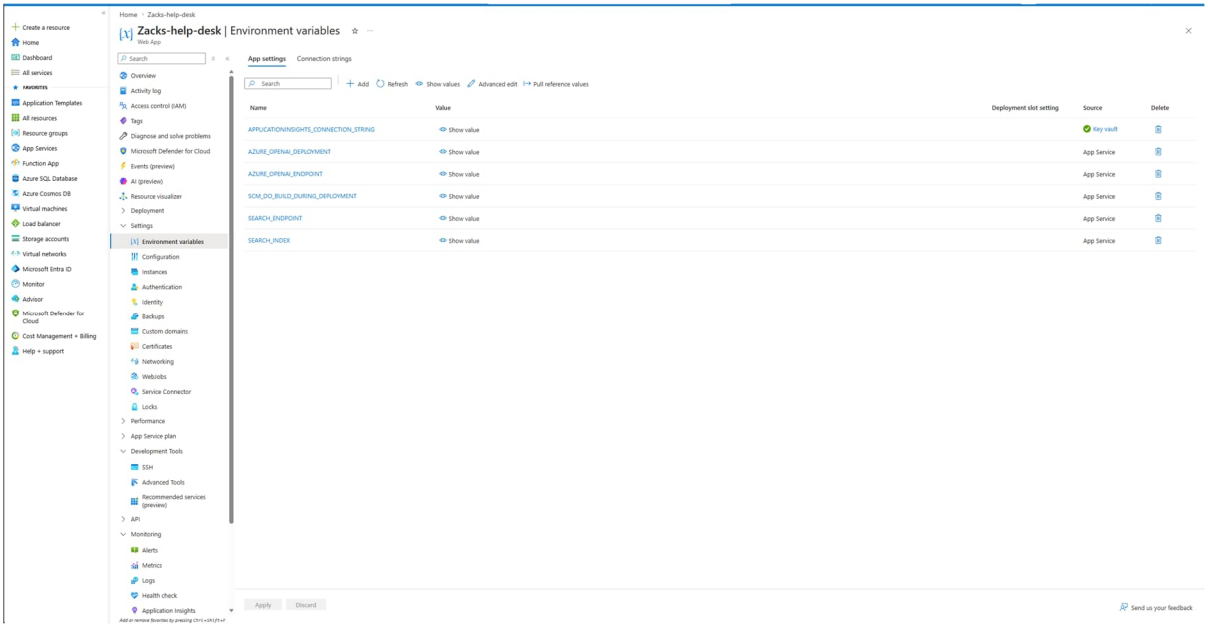

Figure 7. The end state: App Service configuration holding only endpoint addresses — no keys — with the Application Insights connection string sourced from Key Vault (green badge).

Results

The system is live, and every migration objective is verifiable in the running deployment:

- Deployment: a push to main builds the front end and releases the application with no manual steps; failed builds never reach production; a change reaches production in [X] minutes end to end.
- Retrieval: a 50-document, four-format corpus indexed in Azure AI Search with automatic chunking and embeddings; hybrid keyword-plus-vector queries with semantic re-ranking; the index persists across restarts, eliminating v1's cold-start rebuild.
- Generation: gpt-5-mini on Azure OpenAI, grounded in retrieved passages with per-answer source citations, authenticated without a key.
- Observability: request, dependency, and exception telemetry in Application Insights — used during the migration itself to diagnose failures that raw logs truncated.
- Security: zero API keys in code, repository, or configuration; Managed Identity with RBAC for every service call; the residual secret in Key Vault; the old keys regenerated so they cannot silently return — a Zero Trust posture for service-to-service access.
- Cost: the steady-state footprint is a basic App Service tier, an AI Search tier, and per-token model usage measured in single dollars at demonstration traffic — with the identity infrastructure itself costing nothing.
- Code: three pipeline modules ([N] lines) deleted outright; the application is smaller than before the migration while the system around it does strictly more.

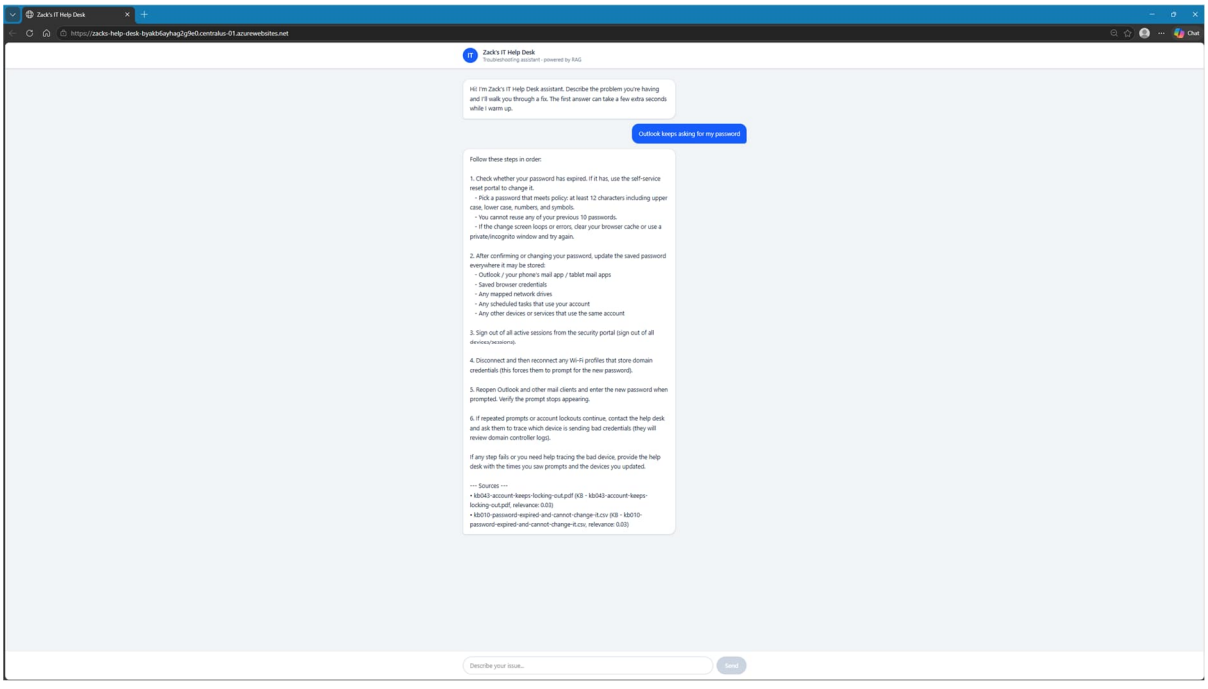

Figure 8. The live system on Azure: a grounded, step-by-step answer with per-source citations, served through the fully migrated, keyless stack.

Core Technologies

- Azure AI Search — the managed retrieval layer: integrated vectorization (automatic chunking and embedding), hybrid search, and semantic re-ranking.
- Azure OpenAI — text-embedding-3-small for vectorization and gpt-5-mini for grounded generation.
- GitHub Actions — CI/CD from repository to App Service, including the front-end build.
- Application Insights (OpenTelemetry) — auto-instrumented request, dependency, and exception telemetry.
- Managed Identity, Entra ID, and Azure RBAC — keyless service-to-service authentication, with DefaultAzureCredential unifying cloud and local development.
- Azure Key Vault — storage for the remaining configuration secret, injected via Key Vault reference without code changes.
- Azure Blob Storage — the knowledge corpus, decoupled from the application release cycle.
- FastAPI and React — the application layer, carried over from v1 largely intact — evidence the migration changed the platform, not the product.

Engineering Concepts Demonstrated

- Incremental re-platforming — migrating a live system service by service, each phase independently verifiable and reversible.
- Managed-service RAG — replacing custom ingestion, chunking, embedding, and vector storage with a configured indexing pipeline, and knowing what the trade buys and costs.
- Data-plane RBAC diagnosis — differential testing across identities, token acquisition and JWT inspection from inside the workload, and reasoning about role scope, inheritance, and propagation.
- Model lifecycle management — navigating marketplace quotas, model retirements, and breaking API changes between model generations as routine engineering inputs rather than surprises.
- Zero-secret architecture — workload identity over shared credentials, least-privilege roles, secrets centralized and referenced rather than copied.
- Observability-driven debugging — instrumenting first, then using structured telemetry to diagnose the remainder of the migration.
- AI-assisted engineering workflow — planning, diagnosis, and code drafted in collaboration with Claude, with human verification gates at every phase boundary.

Known Limitations

Role assignments sit at resource-group scope — expedient during debugging, broader than least privilege prescribes; tightening them to per-resource scope is straightforward now that the working configuration is known. The RBAC diagnosis, while successful, consumed hours partly because role changes on AI resources propagate slowly and cache invisibly; the case study records the diagnostic sequence precisely because the platform gives so little feedback. Deployment goes straight to

production with no staging slot or smoke-test gate. The indexer runs on demand rather than on a schedule, so new knowledge-base uploads require a manual run. Retrieval quality still has no evaluation harness — a known gap carried forward from v1. And the corpus remains demonstration-scale; the ingestion pipeline is the thing being demonstrated, not corpus size.

Business Applications

The v1 case study argued that cited, grounded answers are what make RAG viable in serious settings. This migration supplies the other half of that argument: the operational properties organizations actually procure against.

- Identity-based security — for government and regulated-industry clients, 'no standing credentials, all access via workload identity and auditable RBAC' is frequently a hard requirement, not a preference. The system now meets it. That posture has a name — Zero Trust: no shared credentials, least-privilege roles, and every access decision attributable to an identity and auditable through Entra ID.
- Operational continuity — knowledge updates without redeployment, deployments without a person, telemetry that shows what failed and why: the difference between a demo and a service someone will put their name on.
- Vendor and model agility — the same migration pattern that absorbed two forced model pivots here is what lets an organization exchange models as pricing, quotas, and retirements shift, without rebuilding the application around them.
- A reusable migration playbook — the phase structure (CI/CD, telemetry, data, retrieval, model, identity) applies to any AI prototype an organization wants to industrialize, which is a consulting deliverable in its own right.
- Domain transfer — the same grounded, cited RAG pattern applies directly to environmental and AEC document corpora — site assessment reports, remediation records, permit and compliance documentation — where source citation is a regulatory expectation rather than a convenience.

Future Enhancements

- Staging slot with a health-check gate and slot swap, so releases are verified before they take traffic.
- Least-privilege RBAC — narrowing role scope from resource group to individual resources, and trimming the redundant assignments accumulated during debugging.
- Scheduled indexing and a real corpus — moving the indexer to a schedule and scaling the knowledge base toward the volumes the scenario calls for.
- Retrieval evaluation — a question-to-article test set to measure retrieval quality and catch regressions when chunking or embedding configuration changes.
- Claude on Microsoft Foundry — the original model target, revisited when quota is granted; the generator's isolation makes it a small, contained swap.

- Availability tests and alerting — closing the observability loop so the system reports its own failures before a user does.

Key Takeaways

The most impactful reason to migrate to a managed service is to reduce the dependence on infrastructure that can break, drift, or demand maintenance. Success can be measured in deleted code and every line of handwritten code removed is a step to securing a more reliable pipeline. The engineering judgment being showcased is not writing the pipeline in a completely original way, but rather that recognizing an engineer's role is maximized when owning the pipeline stops being their entire responsibility.

There are some unique challenges that real clouds produce that weren't encountered in previous case studies. Constraints are architectural forces - this isn't a new revelation for building applications. What's new is how those constraints present themselves; a quota of zero, a retired model, and a renamed API parameter each redirected the model migration within a single working session. But still, the plan survived because its phases were independent and its layers isolated - the same modularity argument the previous case studies made, now stress-tested by a platform actively changing underneath the project. Architecture plans that have no ability to adapt turn roadmaps into wishes.

In a cloud, identity debugging is a crucial discipline to hone. Method beats patience. When hand-writing code, there is a legitimate aspect of debugging where you can manipulate lines to isolate issues. In a cloud, when a portal can take anywhere from 5 - 30 minutes to make updates in a background, those same variable manipulations feel more like guessing-and-checking rather than a patient payoff. That loss of visibility demands a more methodical approach tailored to the ecosystem. Live monitoring and logging services are what enable differential testing that produces the evidence needed for a diagnosis.

Finally, the project validates a way of working. Every phase was planned, debugged, and documented in collaboration with AI tooling, backed by a human verification gate at each checkpoint - and the hardest problem in the project was solved by the human-directed experiment, informed by AI-assisted analysis. That division of labor - the assistant accelerating everything, the engineer owning verification and judgment - is the workflow modern AI teams are converging on, and this migration was an end-to-end rehearsal of it.